RED & WHITE SKILL EDUCATION

DEEP LEARNING

PR 2

Project Definition

Topics: MLP · Activation Functions · Weight Initialization · Loss Functions · Batch Normalization · Optimizers

Total Marks: 10

Field	Detail
Institute	Red & White Skill Education
Subject	Deep Learning
Project	PR 2
Total Marks	10
Topics	MLP · Activation Functions · Weight Initialization · Loss Functions · Batch Normalization · Optimizers
Dataset	Adult Income Dataset (Census Income) — UCI Machine Learning Repository
Task Type	Binary Classification — predict whether annual income exceeds $50K
Framework	TensorFlow 2.x / Keras + scikit-learn (Python 3.x, Jupyter Notebook)

§ 1 · Dataset

This project uses the Adult Income Dataset (also known as the Census Income Dataset), extracted from the 1994 US Census database by Barry Becker and donated to the UCI Machine Learning Repository. It contains demographic and employment records for 48,842 individuals, and the task is to predict whether a person earns more than $50,000 per year. The mix of numeric and categorical features — age, education, occupation, hours worked, country of origin — makes this a rich, realistic tabular dataset that demands proper preprocessing before any neural network can be trained on it. This is the kind of real-world structured data that ANN models handle in industry: HR analytics, credit scoring, and income-based targeting.

Field	Detail
Dataset Name	Adult Income Dataset (Census Income)
Source	UCI Machine Learning Repository
UCI URL	https://archive.ics.uci.edu/dataset/2/adult
Kaggle URL	https://www.kaggle.com/datasets/wenruliu/adult-income-dataset
File	adult.csv (recommended — pre-merged train + test split, 48,842 rows)
Total Rows	48,842 records (32,561 original train + 16,281 test — use full file)
Features	14 input features (6 numeric, 8 categorical) + 1 binary target column
Target Column	income — '<=50K' or '>50K' (string; encode as 0 and 1 respectively)
Class Balance	~76% <=50K · ~24% >50K — imbalanced; discuss in EDA
Missing Values	Missing values encoded as '?' in the raw file — requires handling
License	CC BY 4.0 — UCI Machine Learning Repository (1996)

Dataset columns — full reference:

Column / Feature	Type	Values / Range	Description
age	*Numeric*	17 – 90	Age of the individual in years
workclass	*Categorical*	8 categories	Employment type: Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked
fnlwgt	*Numeric*	12,285 – 1,484,705	Final weight — census sampling weight. Drop this column before modelling (it is a survey weight, not a predictive feature)
education	*Categorical*	16 categories	Highest level of education attained (e.g., Bachelors, Masters, HS-grad, Some-college, Doctorate)

education.num	*Numeric*	1 – 16	Numeric encoding of education level. Redundant with 'education' — keep education.num and drop the text 'education' column
marital.status	*Categorical*	7 categories	Marital status: Married-civ-spouse, Divorced, Never-married, Separated, Widowed, Married-spouse-absent, Married-AF-spouse
occupation	*Categorical*	14 categories	Occupation type: Tech-support, Craft-repair, Other-service, Sales, Exec-managerial, Prof-specialty, Handlers-cleaners, Machine-op-inspct, Adm-clerical, Farming-fishing, Transport-moving, Priv-house-serv, Protective-serv, Armed-Forces
relationship	*Categorical*	6 categories	Relationship within family unit: Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried
race	*Categorical*	5 categories	Race: White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black
sex	*Categorical*	2 categories	Sex: Male, Female
capital.gain	*Numeric*	0 – 99,999	Capital gains income (highly skewed — most values are 0)
capital.loss	*Numeric*	0 – 4,356	Capital losses (highly skewed — most values are 0)
hours.per.week	*Numeric*	1 – 99	Average hours worked per week
native.country	*Categorical*	41 categories	Country of origin (highly sparse — group minority countries into 'Other' during preprocessing)
income	*Binary Target*	<=50K or >50K	Target variable — encode as: <=50K = 0, >50K = 1

Note: Download adult.csv from Kaggle (link above) — it is cleaner than the raw UCI version (no trailing periods in the income column). Load with pd.read_csv('adult.csv'). Strip whitespace from all string columns with df[col].str.strip(). Replace '?' with np.nan, then impute or drop. Drop 'fnlwgt' and 'education' (keep 'education.num'). One-hot encode all remaining categorical columns with pd.get_dummies(drop_first=True). Apply StandardScaler to all numeric columns only. Use train_test_split with test_size=0.2, random_state=42, stratify=y.

Business Context: Predicting whether an individual earns above or below the ₹50K threshold directly supports HR analytics, targeted government benefit allocation, and fintech credit-risk scoring — all high-relevance domains for Data Science graduates entering the Indian job market.

§ 2 · Project Tasks — What to Do

Complete ALL tasks in a single Jupyter Notebook named DL_PR2.ipynb. Structure the notebook with Markdown headings matching the task titles below. Each task builds on the previous — do not skip or reorder. Your goal is to understand WHY each design choice is made for tabular ANN training, not just how to write the code.

Task 1: Data Loading, Cleaning & Exploratory Data Analysis

#	Action	Details / Configuration
1.1	**Load and inspect the dataset**	import pandas as pd, numpy as np. df = pd.read_csv('adult.csv'). Print df.shape (should be 48842 × 15), df.dtypes, and df.head(). Strip whitespace from all object columns: for col in df.select_dtypes('object'): df[col] = df[col].str.strip().
1.2	**Handle missing values**	Replace '?' with np.nan: df.replace('?', np.nan, inplace=True). Check df.isnull().sum() — missing values exist in workclass (~1,836), occupation (~1,843), and native.country (~583). Drop rows with any missing value (df.dropna(inplace=True)) — this removes ~2,399 rows, leaving ~46,443 clean records. In a Markdown cell: justify dropping over imputation — the missing columns are categorical with many categories; mode imputation would disproportionately inflate the most common category.
1.3	**Drop redundant columns**	Drop 'fnlwgt' (census sampling weight — not a predictive demographic feature) and 'education' (the text version — 'education.num' already encodes this numerically). df.drop(columns=['fnlwgt','education'], inplace=True). Confirm df.shape is now (≈46,443 × 13).
1.4	**Encode the target**	df['income'] = (df['income'] == '>50K').astype(int). Confirm: df['income'].value_counts() — approximately 0: 35,000 (≈75%) and 1: 11,000 (≈25%). Plot a sns.countplot with bar annotations. Add a Markdown cell: this ~3:1 class imbalance means accuracy alone is misleading (a model predicting '0' always achieves ~75%). We will track F1-score, Precision, and Recall alongside accuracy throughout.
1.5	**Exploratory visualisations**	Generate: (a) sns.histplot of 'age' coloured by 'income' (hue=income). (b) sns.boxplot of 'hours.per.week' by 'income'. (c) sns.countplot of 'education.num' by 'income'. (d) Correlation heatmap of all numeric columns after encoding. Add a Markdown cell per plot: describe what the pattern reveals about income predictors (e.g., higher education.num strongly correlates with >50K income).
1.6	**Encode categoricals & scale numerics**	Separate features: X = df.drop('income', axis=1). y = df['income']. One-hot encode categorical columns: X = pd.get_dummies(X, drop_first=True). Note the final number of columns after OHE (typically ~95–105 columns). Apply StandardScaler to numeric columns only (age, education.num, capital.gain, capital.loss, hours.per.week) — NOT to OHE binary columns. Use train_test_split(X, y, test_size=0.2, random_state=42, stratify=y). Print all split shapes and confirm class ratio is preserved in both splits.

Task 2: MLP Architecture — Baseline ANN

#	Action	Details / Configuration

2.1	**Define the reusable build_ann() function**	Write a Python function build_ann(input_dim, hidden_units=[128, 64], activation='relu', initializer='glorot_uniform', use_batch_norm=False, optimizer='adam', loss='binary_crossentropy') → returns a compiled keras.Sequential model. This function is the foundation of every subsequent experiment — all tasks call build_ann() with one parameter changed at a time. Having a single function enforces controlled experiments.
2.2	**Baseline ANN architecture**	The baseline network (default params): Dense(128, relu, glorot_uniform) → Dense(64, relu, glorot_uniform) → Dense(1, sigmoid). Output is a single sigmoid neuron for binary classification. Print model.summary(). Manually verify total parameters using the input_dim (≈100 after OHE): (input_dim×128+128) + (128×64+64) + (64×1+1). In a Markdown cell: explain WHY sigmoid (not softmax) is used for binary output — sigmoid maps the single logit to a probability in (0,1); for binary tasks, a single output neuron is more efficient than a 2-neuron softmax.
2.3	**Train the baseline**	model_base = build_ann(input_dim=X_train.shape[1]). history_base = model_base.fit(X_train, y_train, epochs=50, batch_size=64, validation_split=0.1, verbose=1). Plot training vs validation loss AND accuracy as a 1×2 subplot. Title: 'Baseline ANN — Training Curves (ReLU · Glorot Uniform · Adam · BCE)'.
2.4	**Evaluate the baseline with full metrics**	y_pred_prob = model_base.predict(X_test).flatten(). y_pred = (y_pred_prob > 0.5).astype(int). Print classification_report(y_test, y_pred). Plot confusion matrix heatmap with income labels (0=<=50K, 1=>50K). Add a Markdown cell: record Precision, Recall, and F1-score for class 1 (>50K) specifically — these matter more than overall accuracy for the minority class. Record all metrics in the results table you will build in Task 7.
2.5	**WHY tabular data suits ANN — Markdown**	Add a Markdown cell: unlike image or sequence data (which need CNN or RNN respectively), structured tabular data with numeric and encoded categorical features is the natural input for a fully connected MLP. Each input neuron receives one scaled feature; the hidden layers learn weighted combinations and non-linear interactions between features like age × education or hours × occupation that a simple logistic regression cannot capture.

Task 3: Activation Functions — Comparison Experiment

#	Action	Details / Configuration
3.1	**Train 4 models with different hidden activations**	Call build_ann() with activation set to each of: 'relu', 'tanh', 'sigmoid', 'elu'. All other parameters at defaults. Train all 4 models for 50 epochs (batch_size=64, validation_split=0.1). Store History objects: history_relu, history_tanh, history_sigmoid, history_elu.
3.2	**Activation comparison — 4-panel plot**	Create a 2×2 subplot figure. Each panel: training vs validation accuracy over 50 epochs for one activation. Use Red #CC0000 for training, Charcoal #2C2C2C for validation curves. Add a horizontal dashed line at 75% accuracy (naive majority-class baseline) on each panel. Annotate each panel title with the final validation accuracy and F1-score for class 1.
3.3	**Dead neuron check — ReLU**	Extract post-activation outputs of the first hidden layer for 500 test samples using a Keras sub-model: intermediate = keras.Model(inputs=model_relu.input, outputs=model_relu.layers[1].output). activations = intermediate.predict(X_test[:500]). Compute: dead_fraction = np.mean(activations == 0). Plot a histogram of mean activation per

		neuron. Title: 'ReLU First-Layer Activation Distribution'. Add a Markdown cell: report dead_fraction and explain how ELU avoids dead neurons by allowing small negative outputs for negative inputs (output = α(e^x − 1) for x < 0), keeping the gradient path alive.
3.4	**Gradient flow check — Sigmoid hidden layers**	On the sigmoid-activated model, use tf.GradientTape to compute gradients of the loss w.r.t. each layer's weights on a single batch. Plot a horizontal bar chart: mean |gradient| per Dense layer (Layer 1, Layer 2, Output). Title: 'Gradient Magnitude by Layer — Sigmoid Hidden Activations'. Add a Markdown cell: if Layer 1 gradients are orders of magnitude smaller than the Output layer, vanishing gradients are occurring — explain WHY (σ'(x) = σ(x)(1−σ(x)) ≤ 0.25 always; multiplied across layers, gradients shrink exponentially).
3.5	**Activation summary Markdown table**	Build a Markdown table: Activation | Formula | Output Range | Zero-Centred | Vanishing Gradient Risk | Dead Neuron Risk | Val Accuracy on Adult (this run). Rows: ReLU, Tanh, Sigmoid (hidden), ELU. Add a final row: Sigmoid (output layer only) — note that sigmoid in the output layer is always correct for binary classification regardless of which activation is used in hidden layers. State your recommendation for this dataset and why.

Task 4: Weight Initialization Techniques

#	Action	Details / Configuration
4.1	**Train 5 models with different initialisers**	Call build_ann(activation='relu') with initializer set to each of: 'glorot_uniform', 'glorot_normal', 'he_uniform', 'he_normal', 'zeros'. All other parameters at defaults. Train all 5 for 50 epochs. Store History objects.
4.2	**WHY initialisation matters — Markdown (write before training)**	Add this Markdown cell before the training code: Neural networks trained with all-zero weights suffer from the symmetry problem — every neuron in a layer produces the same output, receives the same gradient, and updates to the same value. The network collapses to a single effective neuron per layer regardless of width. Xavier/Glorot initialisation draws weights from a distribution with Var = 2/(fan_in + fan_out), calibrated to keep the variance of activations and gradients stable across tanh/sigmoid layers. He initialisation uses Var = 2/fan_in — the extra factor of 2 compensates for ReLU zeroing out half of all activations on average, maintaining the effective variance for ReLU networks.
4.3	**Convergence speed comparison plot**	Plot validation accuracy over 50 epochs for all 5 initialisers on the same axes. Use distinct colour and linestyle per initialiser. Add a horizontal dashed line at 84% as a target benchmark. Mark with vertical tick lines the first epoch each non-zeros model crossed 82% validation accuracy. Title: 'Weight Initialisation — Convergence Speed Comparison (Adult Income Dataset)'. Add a legend. Expected result: zeros stays flat near 75% (majority class); He variants converge fastest with ReLU.
4.4	**Zeros failure demonstration**	Plot the training loss curve of the zeros-initialised model separately, alongside its validation accuracy curve. In a Markdown cell: explain the flat accuracy (~75%) as the network outputting a constant prediction close to 0 (the majority class) for every sample — it never breaks symmetry because all neurons have identical gradients and update identically. This is the clearest demonstration of why weight initialisation is not merely a detail.
4.5	**Weight distribution visualisation**	Before any training, extract initial weights from the first Dense layer for both he_normal and glorot_uniform models. After 50 epochs, extract again. Plot 4 histograms (2×2): he_normal before / he_normal after /

		glorot_uniform before / glorot_uniform after. Title: 'Weight Distributions — Initial vs Trained'. Add a Markdown cell: the initial distributions should closely match their theoretical shapes (narrower for glorot, wider for he). After training, both should have spread and shifted — backpropagation reshapes the distribution; the initialiser only sets the starting point.

Task 5: Loss Functions

#	Action	Details / Configuration
5.1	**Binary Cross-Entropy — the correct loss**	Confirm that the baseline (Task 2) uses loss='binary_crossentropy'. In a Markdown cell: explain the BCE formula — L = −[y·log(p) + (1−y)·log(1−p)] — where p is the sigmoid output and y is the true label (0 or 1). BCE directly penalises confident wrong predictions (when p → 1 but y = 0, log(1−p) → −∞). It is the negative log-likelihood of a Bernoulli distribution — the natural choice for binary classification.
5.2	**MSE as classification loss — wrong tool experiment**	Train model_mse = build_ann(loss='mean_squared_error', activation='relu', initializer='he_normal'). Keep the sigmoid output neuron. Train 50 epochs. Plot validation F1-score (class 1) of model_mse vs model_base on the same axes over epochs. Add a Markdown cell: MSE treats the sigmoid output as a regression target and does not penalise confident wrong predictions as harshly as BCE. Its gradient for the sigmoid output neuron includes a σ(x)(1−σ(x)) term that vanishes at saturation — BCE's gradient is simply (p − y), which is always well-defined. On imbalanced data like Adult Income, this difference is especially visible in minority-class F1.
5.3	**Weighted Binary Cross-Entropy — handling class imbalance**	The Adult Income dataset is ~75% class 0 and ~25% class 1. Compute class weights: from sklearn.utils.class_weight import compute_class_weight. weights = compute_class_weight('balanced', classes=[0,1], y=y_train). class_weight_dict = {0: weights[0], 1: weights[1]}. Train model_wbce = build_ann(loss='binary_crossentropy', activation='relu', initializer='he_normal'). Pass class_weight=class_weight_dict to model.fit(). Plot Precision, Recall, and F1-score for class 1 (>50K) across: model_base (no weight) vs model_wbce (weighted). Add a Markdown cell: weighted BCE penalises misclassifying minority-class samples more heavily, improving Recall for class 1 at the cost of some Precision — crucial in income-targeting applications where missing a high-earner is more costly than a false positive.
5.4	**Focal Loss — advanced imbalance handling**	Implement Focal Loss as a custom Keras loss: def focal_loss(gamma=2.0, alpha=0.25): def loss(y_true, y_pred): y_pred = tf.clip_by_value(y_pred, 1e-7, 1-1e-7); bce = -y_true*tf.math.log(y_pred) - (1-y_true)*tf.math.log(1-y_pred); weight = alpha * y_true * (1-y_pred)**gamma + (1-alpha) * (1-y_true) * y_pred**gamma; return tf.reduce_mean(weight * bce). Train model_focal using this loss. Compare class-1 Recall and F1 with model_wbce and model_base. Add a Markdown cell: Focal Loss was introduced by Facebook AI (Lin et al., 2017) for object detection with extreme class imbalance. The (1−p)^γ factor down-weights easy examples and focuses training on hard, misclassified ones.
5.5	**Loss function summary Markdown table**	Add a Markdown table: Loss Function | Formula (simplified) | Best For | Imbalance Handling | Class-1 F1 on Adult (this run). Cover: BCE, Weighted BCE, MSE (classification), Focal Loss. Final row: recommend which loss to use for the Adult Income ANN in production and justify the choice.

Task 6: Batch Normalization

#	Action	Details / Configuration
6.1	**Build ANN with Batch Normalization**	model_bn = keras.Sequential([keras.layers.Dense(128, kernel_initializer='he_normal', input_shape=(X_train.shape[1],)), keras.layers.BatchNormalization(), keras.layers.Activation('relu'), keras.layers.Dense(64, kernel_initializer='he_normal'), keras.layers.BatchNormalization(), keras.layers.Activation('relu'), keras.layers.Dense(1, activation='sigmoid')]). The canonical order is Dense (linear) → BatchNorm → Activation (non-linear). model_bn.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy']). Print model_bn.summary() — note that BatchNormalization layers each add 4×neurons parameters: gamma (γ), beta (β), moving_mean, moving_variance. Only γ and β are trainable; moving statistics are non-trainable.
6.2	**WHY Batch Normalization — Markdown**	Add a Markdown cell: during training, as the weights of each layer update, the distribution of that layer's inputs (the outputs of the previous layer) shifts. This forces every downstream layer to constantly re-adapt to a moving target — called Internal Covariate Shift (Ioffe & Szegedy, 2015). Batch Normalization addresses this by normalising the pre-activation values within each mini-batch to zero mean and unit variance, then applying learnable scale (γ) and shift (β): BN(x) = γ·((x−μ_B)/√(σ²_B+ε)) + β. At inference, BN uses running statistics (exponential moving averages of batch μ and σ²) accumulated during training — not the current batch. Benefits for tabular data: faster convergence, reduced sensitivity to learning rate, mild regularisation effect (batch statistics add noise).
6.3	**Train and compare: with vs without BatchNorm**	Train model_bn for 50 epochs (batch_size=64, validation_split=0.1, Adam). Compare against model_base (Task 2) on the same figure: 2-panel plot (loss left, accuracy right), 4 lines total. Title: 'Batch Normalization vs Baseline — Training Dynamics (Adult Income)'. Observe: model_bn should show a tighter train–validation gap and faster convergence. Evaluate both on the test set and report Accuracy and F1 (class 1).
6.4	**BatchNorm position experiment**	Build two additional models: (a) model_bn_after — Dense → Activation('relu') → BatchNorm. (b) model_bn_before — Dense → BatchNorm → Activation('relu') (canonical). Train both 30 epochs. Plot validation accuracy. Add a Markdown cell: the original BN paper places normalisation before the activation; many practitioners find 'after activation' performs comparably or better on some datasets. Report which order performed better on Adult Income and whether the difference was statistically meaningful.
6.5	**Inspect learned gamma and beta**	After training model_bn, retrieve the BatchNormalization parameters of the first BN layer: bn_layer = model_bn.layers[1]. gamma, beta, running_mean, running_var = bn_layer.get_weights(). Plot gamma values as a bar chart (128 bars) and beta values as a bar chart below. Add a Markdown cell: neurons where |gamma| is close to zero have been effectively gated off — the network learned their pre-activation distribution has low predictive value for income classification. Neurons with large |gamma| are the model's strongest internal signals.

Task 7: Optimizers — Convergence & Final Comparison

#	Action	Details / Configuration

7.1	**Build 5 models with different optimisers**	Use build_ann(input_dim=X_train.shape[1], activation='relu', initializer='he_normal', use_batch_norm=True, loss='binary_crossentropy') — i.e., the strongest configuration so far. Vary only the optimizer: (a) 'sgd' — vanilla SGD. (b) keras.optimizers.SGD(learning_rate=0.01, momentum=0.9) — SGD with momentum. (c) 'rmsprop'. (d) 'adam' — default (lr=0.001, β_1=0.9, β_2=0.999). (e) keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999) — explicit Adam. Train all 5 for 50 epochs, batch_size=64, validation_split=0.1.
7.2	**Optimiser convergence plot**	Plot validation accuracy over 50 epochs for all 5 optimisers on the same axes. Distinct colour and linestyle per optimiser. Add dashed horizontal reference lines at 80% and 86%. Mark with a vertical arrow the epoch each optimiser first crosses 83% validation accuracy. Title: 'Optimizer Comparison — Validation Accuracy over 50 Epochs (Adult Income ANN)'.
7.3	**WHY Adam dominates — Markdown**	Add a Markdown cell with a structured comparison: (1) Vanilla SGD — fixed global learning rate; slow convergence on sparse, high-dimensional OHE features like the Adult dataset; no per-parameter adaptation. (2) SGD + Momentum — adds velocity term $v = \beta \cdot v + lr \cdot \nabla L$; smooths oscillations and speeds through flat loss regions; still uses the same lr for all parameters. (3) RMSprop — divides lr by a running average of squared gradients per parameter ($\sqrt{E[g^2]}$); adapts to gradient magnitudes; well-suited for non-stationary signals. (4) Adam — combines momentum (first moment m) and RMSprop (second moment v) with bias correction for cold-start (early training when m and v are initialised at 0): $\theta = \theta - lr \cdot \hat{m}/\sqrt{(\hat{v}+\varepsilon)}$. (5) Adam (explicit) — confirms Keras defaults match the original paper (Kingma & Ba, 2015). State which optimiser reached the highest final F1-score for class 1 and which converged fastest.
7.4	**Learning rate sensitivity — SGD vs Adam**	Train SGD models at lr = 0.0001, 0.001, 0.01, 0.1 and Adam models at lr = 0.0001, 0.001, 0.01 (7 models total). Plot validation loss over 30 epochs for each group as a 1×2 figure (SGD lrs left, Adam lrs right). Title: 'Learning Rate Sensitivity — SGD vs Adam'. Add a Markdown cell: Adam is far more robust to lr choice because its adaptive per-parameter learning rates compensate for a suboptimal global lr. SGD diverges or stalls outside a narrow lr window on the Adult dataset's OHE feature space.
7.5	**Final Combined ANN — best configuration**	Build the final model with the best configuration discovered across all 6 tasks: build_ann(input_dim=X_train.shape[1], hidden_units=[128,64], activation='relu', initializer='he_normal', use_batch_norm=True, optimizer='adam', loss='binary_crossentropy' or weighted BCE if it improved F1). Train for 80 epochs with class_weight if applicable. Evaluate on the test set. Record Accuracy, Precision (class 1), Recall (class 1), F1 (class 1), and ROC-AUC.
7.6	**Full results comparison table**	Build a pandas DataFrame with one row per model: Model | Activation | Init | Loss | BatchNorm | Optimizer | Test Acc | Prec(1) | Recall(1) | F1(1) | ROC-AUC. Include: Baseline (Task 2), best Activation (Task 3), best Initialiser (Task 4), Weighted BCE (Task 5), Focal Loss (Task 5), BatchNorm model (Task 6), Final Combined (Task 7). Display with df.style.highlight_max(subset=['F1(1)','ROC-AUC'], color='#ffcccc'). Add ROC curve plot for Baseline, Weighted BCE, and Final Combined — all on the same axes with AUC in the legend. In a Markdown cell: which single technique (activation / init / loss / BN / optimizer) had the largest individual impact on minority-class F1, and why?

§ 3 · Transparent Marking Scheme (10 Marks)

Total Marks: 10 — distributed across 3 components as follows:

Component	Marks	Weight	Deliverable
A. Technical Completion (Jupyter Notebook .ipynb)	5 Marks	50%	Completed notebook — all 7 tasks, Restart & Run All clean
B. Recorded Video Explanation (Face + Screen)	3 Marks	30%	5–10 min MP4/MOV: face + screen, verbal concept explanations
C. GitHub Repository + README.md	2 Marks	20%	Public repo with .ipynb + .html + plots + README + video link
TOTAL	**10 Marks**	**100%**	

NOTE:

- Each **late submission** will **decrease the marks by 1** per **one day**.
- For example, if you are 2 days late submitting the project, your 2 marks will be cut.
- So respect the given deadline, and according to that, submit the project within the given timeline.

Component A: Technical Completion (5 Marks)

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
A1	**0.5**	adult.csv loaded; '?' replaced with NaN; rows with nulls dropped; fnlwgt and education dropped; income encoded 0/1; 4 EDA plots (age hist by income, hours boxplot, education.num countplot, correlation heatmap); get_dummies OHE applied; StandardScaler on numeric columns only; stratified 80/20 split; class imbalance	Loaded, cleaned, encoded, split; 2–3 EDA plots; minor Markdown gaps	Loaded and split; OHE applied; no EDA plots or imbalance discussion	CSV loaded; no cleaning or encoding	Not done

		discussed in Markdown				
A2	**0.5**	build_ann() function defined with all 6 parameters; baseline (128→64→1 sigmoid) built; parameter count verified; sigmoid output explained in Markdown; trained 50 epochs; 1×2 training curves; classification_report + confusion matrix on test set; class-1 F1 recorded in results table; tabular ANN Markdown rationale present	build_ann() defined; baseline trained and evaluated; minor analysis gaps	MLP built without reusable function; accuracy only, no F1	MLP attempted with architecture or compilation errors	No MLP
A3	**1.5**	All 4 activations (ReLU, Tanh, Sigmoid, ELU) trained 50 epochs; 2×2 comparison plot with 75% baseline dashed line and val F1 annotated; dead neuron fraction computed and histogram plotted; per-layer gradient magnitude bar chart for sigmoid model using GradientTape; Markdown summary table (formula, output range, vanishing gradient risk, dead neuron risk, val accuracy on Adult); recommendation with reasoning	4 activations trained and plotted; one diagnostic (dead neurons OR gradient); partial Markdown table	2–3 activations with basic comparison plot; no diagnostics	1 activation only; no comparison	No activation experiment
A4	**1.5**	All 5 initialisers trained (Glorot Uniform, Glorot	4–5 initialisers trained;	3 initialisers; basic convergence	1–2 initialisers; no comparison	No initialiser

		Normal, He Uniform, He Normal, Zeros); convergence plot with epoch-to-82% annotations; zeros failure plot with flat accuracy line and symmetry-problem explanation; weight distribution histograms (He Normal vs Glorot Uniform) before AND after training, 2×2 grid; Markdown: Xavier vs He variance formulas with ReLU and tanh/sigmoid rationale	convergence plot; partial weight distribution or Markdown	plot; no weight distribution		experiment
A5	**1.0**	BCE vs MSE comparison (F1 focus); Weighted BCE with compute_class_weight and Precision/Recall/F1 comparison; Focal Loss custom implementation and evaluation; Loss summary Markdown table; BatchNorm with vs without + position experiment + gamma/beta inspection; 5 optimisers trained; convergence plot with 83% threshold; SGD vs Adam lr sensitivity (7 models); Final Combined model trained 80 epochs; results table (7+ models, 7 metrics); ROC curve (3+	Loss + BatchNorm + Optimiser tasks functional; results table and ROC curve present; one or two sub-steps missing	2 of 3 topics (Loss / BN / Optimiser) functional; partial results table	1 topic partially implemented	None

		models with AUC); Restart & Run All clean				

Subtotal A: 0.5 + 0.5 + 1.5 + 1.5 + 1.0 = 5.0 Marks

Component B: Recorded Video — Face + Screen (3 Marks)

Record a video showing your face (webcam picture-in-picture) and your Jupyter Notebook screen simultaneously. Explain concepts verbally — do not just run cells without speaking.

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
B1	**1.0**	Clear audio and video throughout; face visible via PiP; notebook cell outputs, model summaries, and all plots clearly visible on screen; all 7 tasks demonstrated in order	Clear audio; face mostly visible throughout	Audio unclear; face partially visible	Very poor quality, hard to follow	No video
B2	**1.5**	Verbally explains all 6 core concepts with reasoning: (1) WHY '?' is dropped rather than imputed, and WHY StandardScaler is applied to numeric columns only — not OHE binary columns; (2) WHY sigmoid (not softmax) for binary output, and WHY F1-score matters more than accuracy on this imbalanced dataset; (3) Activation functions — dead neuron mechanism in ReLU, vanishing gradient mechanism in sigmoid, with	Explains 5–6 concepts with reasoning	Demonstrates steps without explaining theory	Reads code from screen; no conceptual depth	No verbal explanation

		formula-level explanation ($\sigma'(x) \leq 0.25$); (4) Weight initialisation — symmetry problem of zeros init; WHY He uses 2/fan_in for ReLU; WHY Xavier uses 2/(fan_in+fan_out) for tanh/sigmoid; (5) Loss functions — WHY BCE beats MSE for classification (gradient at saturation); WHY class weighting and Focal Loss improve minority-class Recall; (6) Batch Normalisation — Internal Covariate Shift, training vs inference behaviour, what gamma and beta learn; (7) Optimisers — Adam β_1 and β_2 explained; WHY Adam is more robust than SGD on high-dimensional OHE feature spaces				
B3	**0.5**	5–10 min duration; covers all 7 tasks; shows build_ann() function definition live; shows all comparison plots (activation 2×2, initialiser convergence, loss F1 comparison, BatchNorm dynamics, optimiser curves, ROC curve); shows	4–5 min; covers most tasks	2–4 min; misses key experiments	Under 2 min or very incomplete	Not submitted

		final results table with highlighted best model				

Subtotal B: 1.0 + 1.5 + 0.5 = 3.0 Marks

Video Requirements

Requirement	Detail
Format	MP4 or MOV, maximum 500 MB
Face visibility	Webcam picture-in-picture overlay visible throughout the full recording
Screen content	Jupyter Notebook — show build_ann() function, model.summary(), all comparison plots, results table, ROC curve
Duration	5 to 10 minutes
Naming convention	DL_PR2_YourName_GRID.mp4
Upload location	Google Drive (Anyone with link can view) or YouTube (Unlisted)
Link placement	Paste the video URL in your GitHub README.md under the Video section

Component C: GitHub Repository + README.md (2 Marks)

Create a public GitHub repository specifically for this project.

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
C1	**0.5**	Repo contains: DL_PR2.ipynb + DL_PR2.html + /plots folder with all key figures clearly named (eda_class_balance.png, activation_comparison.png, initialiser_convergence.png, weight_distributions.png, loss_f1_comparison.png, batchnorm_dynamics.png, optimiser_convergence.png, roc_curves.png, results_table.pn	Key files present; a few plots missing or unnamed	Only .ipynb uploaded; no /plots folder	Repo exists but empty or broken link	No GitHub repo

		g) + requirements.txt				
C2	**1.0**	README contains: project title + dataset name and UCI/Kaggle URL + preprocessing steps (handling '?', OHE, StandardScaler) + all 6 topics with 2-line descriptions + build_ann() parameter table + /plots screenshots with captions + full results table (7+ models, 7 metrics) + video link + tools section	Title + dataset + partial topic docs; missing some plots or results	Title and URL only; no topic descriptions or results	Blank README or single line	No README
C3	**0.5**	3+ meaningful commits, e.g.: 'Add data cleaning — handle missing values, OHE, StandardScaler, stratified split'; 'Build activation comparison — ReLU dead neuron and Sigmoid gradient diagnostics'; 'Add BatchNorm experiment — covariate shift analysis and gamma/beta inspection'	2–3 adequate commits with basic messages	1 bulk commit with all files at once	Commit messages unclear or auto-generated	No commits or single initial commit

Subtotal C: 0.5 + 1.0 + 0.5 = 2.0 Marks

§ 4 · Submission Checklist

Tick every item below before submitting. All items are required for full marks.

- ☐ adult.csv downloaded from UCI / Kaggle URL specified in § 1
- ☐ Whitespace stripped from all string columns using str.strip()
- ☐ '?' replaced with np.nan; df.isnull().sum() checked per column
- ☐ Rows with any null value dropped; df.shape confirmed (~46,443 × 13 after drops)

- ☐ 'fnlwgt' and 'education' columns dropped; 'education.num' retained
- ☐ income column encoded: '>50K' = 1, '<=50K' = 0
- ☐ Class distribution countplot with bar annotations generated
- ☐ Markdown: class imbalance (~75/25%) noted; F1-score chosen as primary minority-class metric
- ☐ 4 EDA visualisations: age histogram by income, hours.per.week boxplot, education.num countplot, correlation heatmap
- ☐ pd.get_dummies(drop_first=True) applied to all categorical columns
- ☐ StandardScaler applied to numeric columns only (age, education.num, capital.gain, capital.loss, hours.per.week) — NOT to OHE binary columns
- ☐ Stratified train_test_split: test_size=0.2, random_state=42, stratify=y; class ratio confirmed in both splits
- ☐ build_ann() function defined with 6 parameters: input_dim, hidden_units, activation, initializer, use_batch_norm, optimizer, loss
- ☐ Baseline ANN: Dense(128, relu) → Dense(64, relu) → Dense(1, sigmoid); parameter count manually verified
- ☐ Markdown: WHY sigmoid (not softmax) for binary output — single logit → probability in (0,1)
- ☐ Baseline trained 50 epochs; 1×2 training curves (loss + accuracy) plotted
- ☐ classification_report and confusion matrix heatmap on test set
- ☐ Precision, Recall, and F1-score for class 1 (>50K) recorded in results table
- ☐ Markdown: WHY tabular MLP is appropriate for structured demographic/employment data
- ☐ All 4 hidden-layer activations trained: ReLU, Tanh, Sigmoid, ELU — 50 epochs each
- ☐ 2×2 comparison subplot with 75% majority-class baseline dashed line; val F1 annotated per panel
- ☐ Dead neuron fraction computed for ReLU; first-layer activation distribution histogram plotted
- ☐ Per-layer gradient magnitude bar chart for sigmoid model (using tf.GradientTape on one batch)
- ☐ Markdown summary table: Activation | Formula | Output Range | Zero-Centred | Vanishing Grad Risk | Dead Neuron Risk | Val Accuracy
- ☐ Recommendation: best activation for Adult Income ANN with reasoning
- ☐ All 5 initialisers trained: Glorot Uniform, Glorot Normal, He Uniform, He Normal, Zeros — 50 epochs each
- ☐ Convergence plot: validation accuracy over 50 epochs for all 5; epoch-to-82% annotation per initialiser
- ☐ Zeros failure plot: flat ~75% accuracy line confirming symmetry problem
- ☐ Markdown: symmetry problem explained; Xavier variance formula (2/(fan_in+fan_out)) vs He formula (2/fan_in)
- ☐ Weight distribution 2×2 histograms: He Normal vs Glorot Uniform — before training AND after 50 epochs
- ☐ BCE vs MSE experiment: validation F1 (class 1) comparison plot over 50 epochs
- ☐ Markdown: BCE gradient formula (p−y) vs MSE gradient with sigmoid factor; WHY BCE is correct for classification
- ☐ compute_class_weight('balanced') used to compute class_weight_dict; model_wbce trained and evaluated
- ☐ Precision / Recall / F1 (class 1) comparison: model_base vs model_wbce — plotted as grouped bar chart
- ☐ Focal Loss custom function implemented: focal_loss(gamma=2.0, alpha=0.25); model_focal trained and evaluated
- ☐ Loss function Markdown summary table: Formula | Label Format | Imbalance Handling | Class-1 F1 on Adult
- ☐ BatchNorm ANN built: Dense → BatchNorm → Activation (canonical order); model.summary() shows trainable vs non-trainable params
- ☐ Markdown: Internal Covariate Shift definition; training (batch stats) vs inference (running mean/var) explained
- ☐ With vs without BatchNorm: 2-panel training dynamics figure (loss + accuracy, 4 lines total)
- ☐ BatchNorm position experiment: before-activation vs after-activation — validation accuracy comparison
- ☐ Gamma and beta bar charts (128 bars each) from first BN layer; gated neurons identified in Markdown
- ☐ All 5 optimiser variants trained with ReLU + He Normal + BatchNorm + BCE configuration
- ☐ Convergence plot: validation accuracy over 50 epochs; 80% and 86% dashed reference lines; epoch-to-83% arrows

- ☐ Markdown: SGD vs Momentum vs RMSprop vs Adam — formula-level comparison; β_1 and β_2 roles explained
- ☐ SGD lr sensitivity: 4 lrs (0.0001, 0.001, 0.01, 0.1); Adam lr sensitivity: 3 lrs (0.0001, 0.001, 0.01); 1×2 figure
- ☐ Final Combined model trained 80 epochs with best configuration; test Accuracy, Precision, Recall, F1, ROC-AUC recorded
- ☐ Full results table: 7+ models × 7 metrics (Acc, Prec, Recall, F1, ROC-AUC, BatchNorm, Optimizer)
- ☐ Highest F1(1) and ROC-AUC highlighted using df.style.highlight_max
- ☐ ROC curve: Baseline + Weighted BCE + Final Combined on same axes with AUC in legend
- ☐ Markdown: which single technique had the largest impact on minority-class F1 and why
- ☐ Every task has opening Markdown cell: title + objective (1 sentence) + expected output
- ☐ All plots: descriptive title, labelled axes, legend, consistent styling
- ☐ Kernel restarted — Restart & Run All executed with zero errors before submission
- ☐ Notebook exported as HTML — both DL_PR2.ipynb and DL_PR2.html present in repository
- ☐ requirements.txt: tensorflow>=2.12.0, scikit-learn>=1.4.0, pandas, numpy, matplotlib, seaborn
- ☐ Video: face (PiP) + Jupyter screen, 5–10 min, MP4/MOV, named DL_PR2_YourName_GRID.mp4
- ☐ Video explains all 6 concepts verbally: preprocessing rationale, sigmoid output, activation diagnostics, initialiser theory, BCE vs Focal Loss, BatchNorm covariate shift, Adam β_1/β_2
- ☐ GitHub: public repository with DL_PR2.ipynb + DL_PR2.html + /plots folder + requirements.txt
- ☐ README.md: dataset URL + 6 topic descriptions + build_ann() table + plots with captions + results table + video link
- ☐ 3+ meaningful Git commits with descriptive messages referencing specific experiments
- ☐ Video link pasted in GitHub README.md under the Video section

Red & White Skill Education · Since 2008 · "Shaping Skills for Scaling Higher...!!!"

© 2026 Red & White Skill Education. All Rights Reserved.